

Generate keys for encryption and decryption

08/12/2022

Creating and managing keys is an important part of the cryptographic process. Symmetric algorithms require the creation of a key and an initialization vector (IV). You must keep this key secret from anyone who shouldn't decrypt your data. The IV doesn't have to be secret but should be changed for each session. Asymmetric algorithms require the creation of a public key and a private key. The public key can be made known to anyone, but the decrypting party must only know the corresponding private key. This section describes how to generate and manage keys for both symmetric and asymmetric algorithms.

Symmetric Keys

The symmetric encryption classes supplied by .NET require a key and a new IV to encrypt and decrypt data. A new key and IV is automatically created when you create a new instance of one of the managed symmetric cryptographic classes using the parameterless `Create()` method. Anyone that you allow to decrypt your data must possess the same key and IV and use the same algorithm. Generally, a new key and IV should be created for every session, and neither the key nor the IV should be stored for use in a later session.

To communicate a symmetric key and IV to a remote party, you usually encrypt the symmetric key by using asymmetric encryption. Sending the key across an insecure network without encryption is unsafe because anyone who intercepts the key and IV can then decrypt your data.

The following example shows the creation of a new instance of the default implementation class for the [Aes](#) algorithm:

C#

```
Aes aes = Aes.Create();
```

The execution of the preceding code generates a new key and IV and sets them as values for the **Key** and **IV** properties, respectively.

Sometimes you might need to generate multiple keys. In this situation, you can create a new instance of a class that implements a symmetric algorithm. Then, create a new key and IV by

calling the `GenerateKey` and `GenerateIV` methods. The following code example illustrates how to create new keys and IVs after a new instance of the symmetric cryptographic class has been made:

C#

```
Aes aes = Aes.Create();  
aes.GenerateIV();  
aes.GenerateKey();
```

The execution of the preceding code creates a new instance of `Aes` and generates a key and IV. Another key and IV are created when the `GenerateKey` and `GenerateIV` methods are called.

Asymmetric Keys

.NET provides the [RSA](#) class for asymmetric encryption. When you use the parameterless `Create()` method to create a new instance, the [RSA](#) class creates a public/private key pair. Asymmetric keys can be either stored for use in multiple sessions or generated for one session only. While you can make the public key available, you must closely guard the private key.

A public/private key pair is generated when you create a new instance of an asymmetric algorithm class. After creating a new instance of the class, you can extract the key information using the [ExportParameters](#) method. This method returns an [RSAParameters](#) structure that holds the key information. The method also accepts a Boolean value that indicates whether to return only the public-key information or to return both the public-key and the private-key information.

You also can use other methods to extract the key information, such as:

- [RSA.ExportRSAPublicKey](#)
- [RSA.ExportRSAPrivateKey](#)
- [AsymmetricAlgorithm.ExportSubjectPublicKeyInfo](#)
- [AsymmetricAlgorithm.ExportPkcs8PrivateKey](#)
- [AsymmetricAlgorithm.ExportEncryptedPkcs8PrivateKey](#)

You can use the [ImportParameters](#) method to initialize an `RSA` instance to the value of an `RSAParameters` structure. Or you can use the [RSA.Create\(RSAParameters\)](#) method to create a new instance.

Never store asymmetric private keys verbatim or as plain text on the local computer. If you need to store a private key, you must use a key container. For more information about how to store a private key in a key container, see [How to: Store Asymmetric Keys in a Key Container](#).

The following code example creates a new instance of the `RSA` class, creates a public/private key pair, and saves the public key information to an `RSAParameters` structure:

C#

```
//Generate a public/private key pair.  
RSA rsa = RSA.Create();  
//Save the public key information to an RSAParameters structure.  
RSAParameters rsaKeyInfo = rsa.ExportParameters(false);
```

See also

- [Encrypting Data](#)
- [Decrypting Data](#)
- [Cryptographic Services](#)
- [How to: Store Asymmetric Keys in a Key Container](#)
- [Cross-Platform Cryptography](#)
- [ASP.NET Core Data Protection](#)